CRUD-операции с использованием Prisma

Полный Product API без ручных INSERT, SELECT, UPDATE и DELETE

Результаты занятия

- Связывать REST и CRUD с методами Prisma Client.
- Реализовывать create(), findMany(), findUnique(), update() и delete().
- Применять where, data, select, orderBy и include.
- Перевести Product Service с pg на Prisma без изменения публичных маршрутов.
- Сохранить валидацию и HTTP-ответы 201, 400, 404, 500.

Учебные вопросы

№	Тема	Содержание
1	CRUD в Prisma Client	Соответствие REST, CRUD и ORM
2	Create	Создание товара и проверка входных данных
3	Read	Получение, фильтрация, сортировка и связи
4	Update и Delete	Изменение, удаление и ошибки
5	Полный Product Service	Замена pg, интеграция и проверка REST API

CRUD в Prisma Client

Каждая операция работы с данными имеет метод модели

CRUD после подключения Prisma

- PostgreSQL по-прежнему хранит данные и проверяет ограничения.
- Prisma Client предоставляет методы модели Product.
- Меняется слой доступа к данным — Product Service, а не HTTP-маршруты.
- Route и Controller продолжают отвечать за маршрутизацию и HTTP.

REST → CRUD → Prisma

HTTP	CRUD	Prisma	Назначение
POST	Create	create()	Создать товар
GET	Read	findMany()	Получить список
GET /:id	Read	findUnique()	Найти товар
PATCH /:id	Update	update()	Обновить
DELETE /:id	Delete	delete()	Удалить

Пять основных методов Product

Чтение

- findMany() → массив объектов или []
- findUnique() → объект или null

Изменение

- create() → созданная запись
- update() → обновлённая запись
- delete() → удалённая запись

REST CRUD → методы Prisma Client

POST → **Create**
Создание

`prisma.product.create()`

GET → **Read**
Чтение (получение)

`findMany() / findUnique()`

Product
Ресурс
(модель)

PATCH → **Update**
Обновление

`update()`

DELETE → **Delete**
Удаление

`delete()`

1 **HTTP** работает между клиентом и API



Клиент

API
(ваше приложение)

2 **Prisma** работает между **Service** и PostgreSQL



Service
(бизнес-логика)

Prisma
(ORM)

PostgreSQL
(база данных)

Параметры Prisma-запроса

```javascript
await prisma.product.update({
  where: { id: 15 },
  data: { price: 42000 }
});
```

Что важно

- where задаёт уникальное условие выбора записи.
- data задаёт новые значения; orderBy сортирует; select выбирает поля; include загружает связи.

Что возвращают методы Prisma

Метод	Результат	Если нет записи
findMany()	Массив	[]
findUnique()	Объект	null
create()	Объект	Ошибка при нарушении ограничений
update()	Объект	P2025
delete()	Объект	P2025

Create

Создаём новый Product через объект data

Метод create()

- Добавляет одну новую строку в PostgreSQL и возвращает созданную запись.
- data содержит поля новой записи; id генерируется автоматически.
- categoryId обязателен и должен ссылаться на существующую строку categories.
- Ограничения PostgreSQL действуют и при работе через Prisma.

Создание Product

```javascript
const product = await prisma.product.create({
  data: {
    name: "SSD 2TB",
    price: 65000,
    categoryId: 2,
    inStock: true
  }
});
```

Что важно

- Prisma выбирает модель через prisma.product.
- Используйте существующий id категории: products.category_id NOT NULL.
- При неверном внешнем ключе операция завершится ошибкой.

Create в Product Service

```javascript
export async function addProduct(data) {
  return prisma.product.create({
    data: {
      name: data.name,
      price: data.price,
      categoryId: data.categoryId,
      inStock: data.inStock ?? true
    }
  });
}
```

Что важно

- Service получает проверенные значения; HTTP req/res здесь отсутствуют.
- Контроллер отвечает за статус 201 Created.

БОЛЬШИЕ ЗНАНИЯ НАЧИНАЮТСЯ С ПРАКТИКИ

POST /api/products → prisma.product.create()

EXPRESS
PRISMA
POSTGRESQL
BETTER DEVELOPERS
BRIGHTER TOMORROW

Создание товара в Express-приложении через Prisma

Клиент
(браузер / приложение)



JSON клиента

```
{
  "name": "Gaming Keyboard",
  "price": 28000,
  "categoryId": 1,
  "inStock": true
}
```

Validation



- Проверка схемы
- Типы данных
- Обязательные поля
- Корректные значения

Controller



Обрабатывает HTTP-запрос, вызывает сервис

products.controller.ts

addProduct(data)



Бизнес-логика (сервис)

products.serroller.ts

prisma.product.create({ data })



Вызов Prisma ORM для создания записи

Prisma ORM

Prisma Client



Генерируемый клиент для работы с БД (type-safe)

PostgreSQL



Сохранение данных в таблицу products

Данные записываются в PostgreSQL, и созданная запись возвращается обратно

созданная запись

```
{
  "id": 42,
  "name": "Gaming Keyboard",
  "price": 28000,
  "categoryId": 1,
  "inStock": true,
  "createdAt": "2025-04-24T10:30:00Z",
  "updatedAt": "2025-04-24T10:30:00Z"
}
```

Controller

Получает созданную запись и формирует HTTP-ответ

products.controller.ts

201 Created
HTTP-ответ клиенту

```
{
  "id": 42,
  "name": "Gaming Keyboard",
  "price": 28000,
  "categoryId": 1,
  "inStock": true,
  "createdAt": "2025-04-24T10:30:00Z",
  "updatedAt": "2025-04-24T10:30:00Z"
}
```



HTTP работает между клиентом и API, Prisma — между Service и PostgreSQL.

РАЗНЫЕ УРОВНИ.
ОДНА ЭКОСИСТЕМА. БОЛЬШИЕ ВОЗМОЖНОСТИ.

Controller создания товара

```javascript
export async function createProduct(req, res, next) {
  try {
    const product = await addProduct(req.body);
    res.status(201).json(product);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- HTTP-слой читает req.body и возвращает HTTP 201.
- Валидация выполняется до обращения к Product Service.

Создание товара через REST API

Запрос: POST /api/products

Код • JSON

```
{
  "name": "SSD 2TB",
  "price": 65000,
  "categoryId": 2,
  "inStock": true
}
```

Результат • JSON

```
{
  "id": 21,
  "name": "SSD 2TB",
  "price": 65000,
  "categoryId": 2,
  "inStock": true
}
```

Пояснение

- Клиент отправляет JSON.
- Validation проверяет поля.
- Service вызывает prisma.product.create().
- Controller отвечает HTTP 201.
- Числа id и categoryId приведены для примера; категория 2 должна существовать.

Validation остаётся необходимой

- ORM не заменяет проверку HTTP-запроса.
- name — непустая строка; price — положительное целое число.
- categoryId — положительное целое число существующей категории.
- inStock — логическое значение, а не строка "true".
- Некорректный запрос должен получить 400, а не 500.

Пример проверки POST: HTTP 400

JAVASCRIPT

```
export function validateCreate(req, res, next) {
  const { name, price, categoryId, inStock } = req.body;
  if (typeof name !== "string" || !name.trim() ||
      !Number.isInteger(price) || price <= 0 ||
      !Number.isInteger(categoryId) || categoryId <= 0 ||
      (inStock !== undefined && typeof inStock !== "boolean")) {
    return res.status(400).json({ message: "Invalid product data" });
  }
  next();
}
```

- Проверяем типы и диапазоны до вызова Prisma.
- Отдельно нужно запрещать неизвестные поля и применять существующие правила валидации проекта.

Read

Получаем список, одну запись и связанную категорию

findMany(): список товаров

```javascript
const products = await prisma.product.findMany({
  orderBy: { id: "asc" }
});
```

Что важно

- Возвращает массив, даже если записей нет.
- orderBy задаёт сортировку без SQL ORDER BY.

Фильтрация через where

```javascript
const products = await prisma.product.findMany({
  where: {
    price: { gte: 10000 },
    inStock: true
  },
  orderBy: { price: "asc" }
});
```

Что важно

- gte означает «больше или равно».
- Условия price и inStock объединяются по AND.

findUnique(): один товар

```javascript
const product = await prisma.product.findUnique({
  where: { id: 15 }
});
```

Что важно

- Условие использует уникальный id.
- Если товара нет, Prisma вернёт null, не HTTP 404.

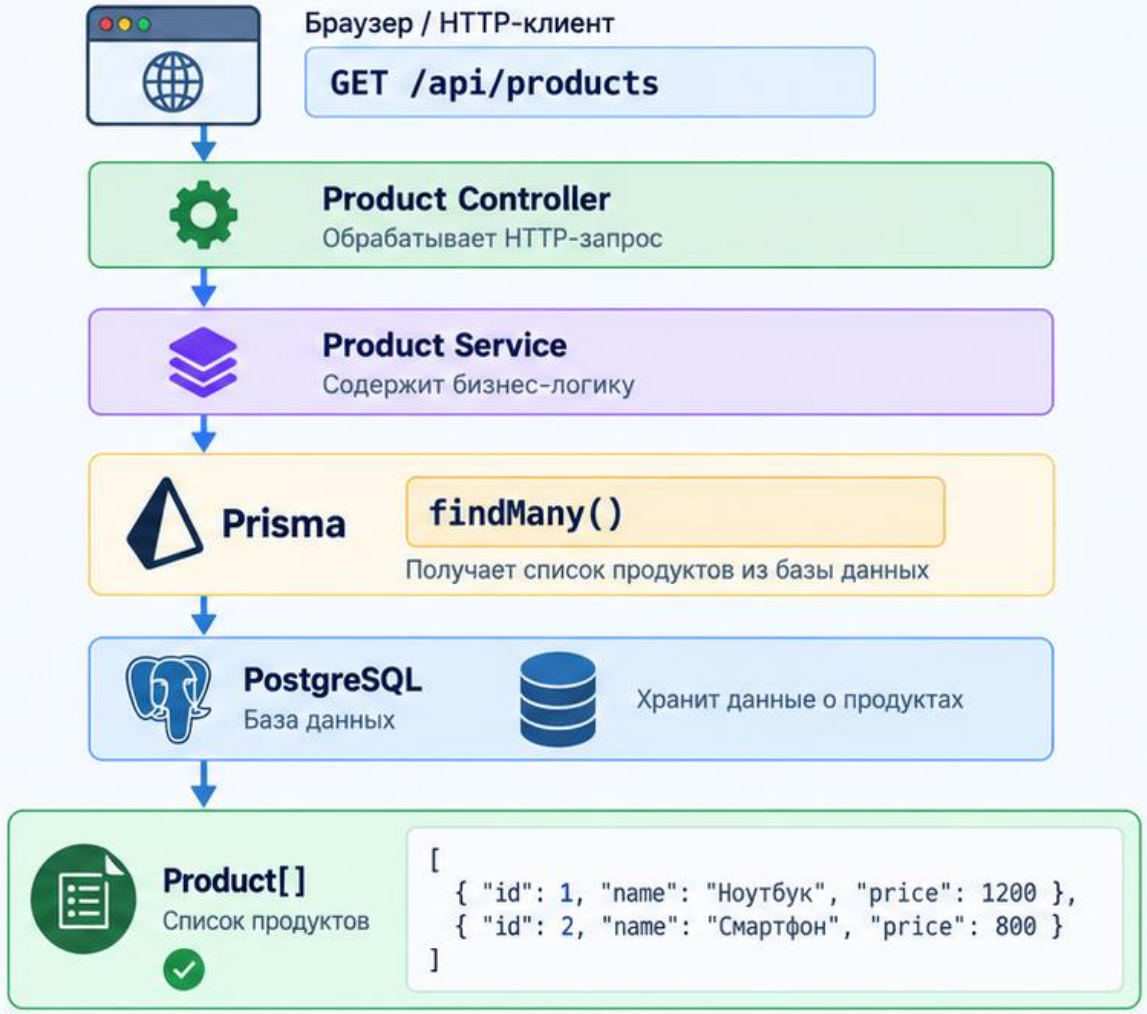
null превращается в HTTP 404

- Prisma не управляет HTTP-статусами.
- Service возвращает Product либо null.
- Controller проверяет отсутствие записи и отвечает 404.
- id из req.params — строка; валидируем и преобразуем его в число.

GET: findMany() и findUnique()

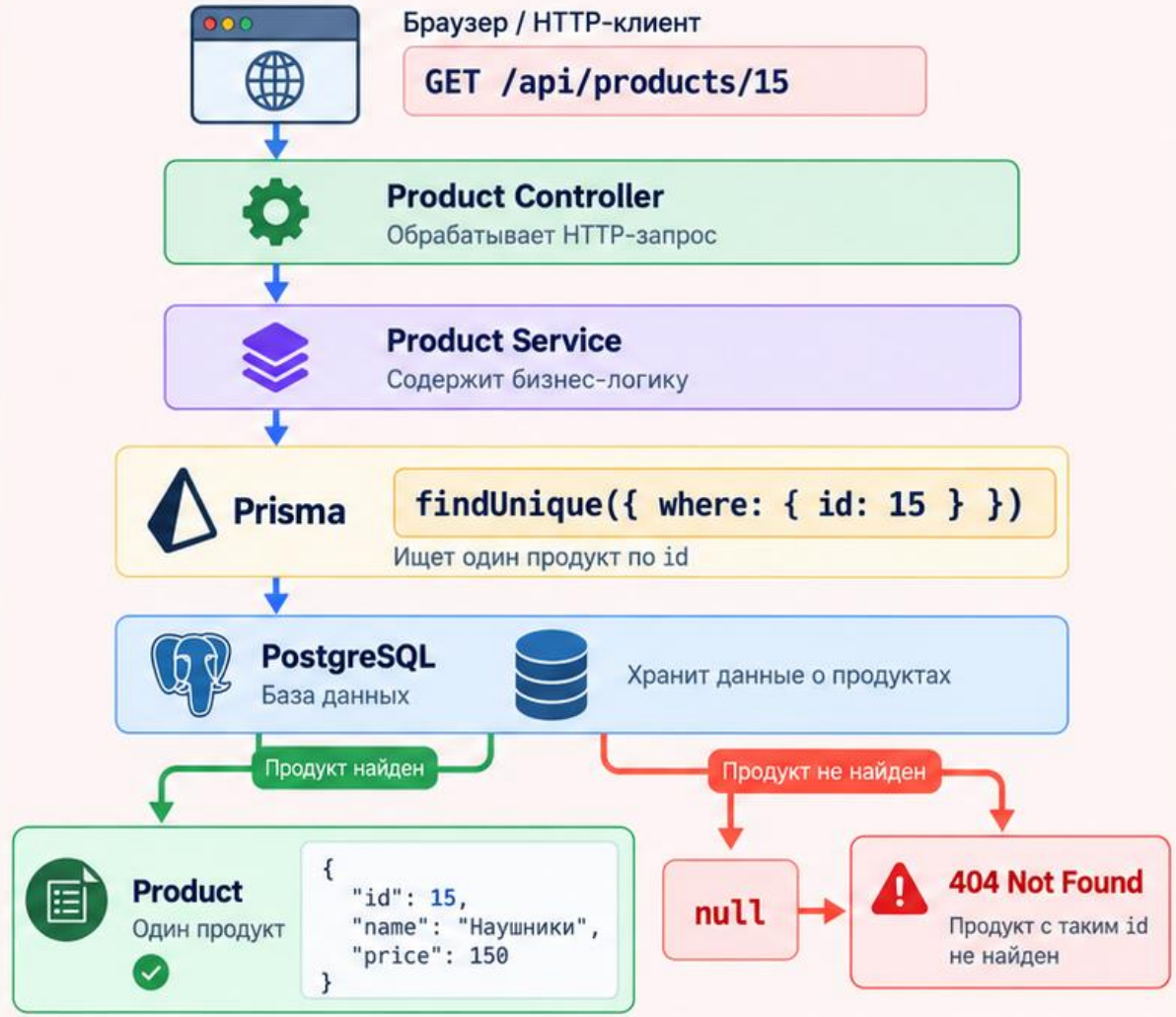
Получение списка продуктов (коллекция)

`findMany()` возвращает список (массив) продуктов.



Получение одного продукта по id

`findUnique()` возвращает один объект или `null`, если продукт не найден.



findMany() возвращает массив, **findUnique()** — один объект или **null**.

Выбираем только нужные поля

```javascript
const products = await prisma.product.findMany({
  select: {
    id: true,
    name: true,
    price: true,
    inStock: true
  }
});
```

Что важно

- select возвращает только перечисленные поля.
- При select неиспользуемые свойства не включаются в объект.

findMany() и findUnique()

findMany()

- Коллекция Product[]
- Фильтрация и сортировка
- Пустой результат: []

findUnique()

- Один уникальный ресурс
- where: { id }
- Пустой результат: null

Связь Product → Category через include

```javascript
const products = await prisma.product.findMany({
  include: { category: true },
  orderBy: { id: "asc" }
});
```

Что важно

- Таблица products хранит category_id, а categories — name.
- include добавляет связанный объект category с его полем name.
- В отличие от select, include используется для получения связанной модели.

Как выглядит результат include

Запрос: GET /api/products

Код • JAVASCRIPT

```
await prisma.product.findMany({
  include: { category: true }
});
```

Результат • JSON

```
[{
  "id": 1,
  "name": "SSD 2TB",
  "price": 65000,
  "categoryId": 2,
  "inStock": true,
  "category": { "id": 2, "name":
"Storage" }
}]
```

Пояснение

- Prisma загружает продукты
- Добавляет объект связанной категории.
- Controller возвращает JSON.
- Если старый API ожидает category как строку, Service должен преобразовать результат.

Update и Delete

Обновляем и удаляем один объект по id

Метод update()

- where выбирает уникальный товар по id.
- data содержит только изменяемые поля.
- Остальные значения сохраняются.
- При отсутствии строки операция выдаёт P2025.

Изменяем цену Product

```javascript
const product = await prisma.product.update({
  where: { id: 15 },
  data: { price: 42000 }
});
```

Что важно

- Меняется только price — это соответствует PATCH.
- Результат содержит новую версию товара.

PATCH и объект changes

- PATCH передаёт только изменяемые свойства.
- Validation должна разрешать только name, price, categoryId, inStock.
- Не переданные поля изменять не требуется.
- Перед прямой передачей data: changes необходимо проверить типы и состав полей.

Update в Product Service

```javascript
export async function updateProductById(id, changes) {
  return prisma.product.update({
    where: { id },
    data: changes
  });
}
```

Что важно

- В примере changes уже прошёл валидацию и преобразование типов.
- id тоже должен быть проверенным целым числом.
- Вариант не требует предварительного SELECT.

Слайд №32

PATCH /api/products/:id → prisma.product.update()

Express.js + Prisma + PostgreSQL

Частичное обновление товара через Prisma Client

Клиент



PATCH /api/products/15

```
{
  "price": 42000
}
```

HTTP-запрос (JSON)

Controller

productController.js

```
export async function
 updateProduct(req, res, next) {
  try {
   const id = Number(req.params.id);
   const product = await
   updateProductById(id, req.body);

   if (!product) {
    return next(new AppError(
     "Product not found", 404
    });
   }

   res.json(product);
  } catch (error) {
   next(error);
  }
}
```

Service

productService.js

```
export async function
 updateProductById(id, changes) {
  const existing =
   await prisma.product.findUnique({
    where: { id }
  });

  if (!existing) {
   return null;
  }

  return prisma.product.update({
   where: { id },
   data: changes
  });
}
```

Prisma Client

```
prisma.product.update()

prisma.product.update({
  where: { id: 15 },
  data: { price: 42000 }
})
```

PostgreSQL

Таблица products

id	name	price	...
15	Laptop	~~35000~~	...

↓

15	Laptop	42000	...

Обновление записи в PostgreSQL

Результат

200 OK

JSON-ответ

```
{
  "id": 15,
  "name": "Laptop",
  "price": 42000,
  "categoryId": 1,
  "inStock": true
}
```

Клиент отправляет PATCH с новыми данными → **Controller** извлекает id и body, вызывает Service → **Service** проверяет существование и вызывает update() → **Prisma Client** выполняет обновление в PostgreSQL → **PostgreSQL** сохраняет изменения в таблице products → **Controller** возвращает обновлённый Product (200 OK)

Важно: PATCH обновляет только переданные поля. Остальные поля остаются без изменений.

Если Product не существует

- findUnique() возвращает null.
- update() и delete() выбрасывают ошибку Prisma с кодом P2025.
- Учебный вариант: сначала findUnique(), затем update/delete.
- Практический вариант: единый errorHandler преобразует P2025 в HTTP 404.
- Предварительное чтение не устраняет гонку: запись могут удалить между запросами.

Вариант с предварительным findUnique()

```javascript
export async function updateProductById(id, changes) {
  const existing = await prisma.product.findUnique({
    where: { id }
  });
  if (!existing) return null;
  return prisma.product.update({
    where: { id },
    data: changes
  });
}
```

Что важно

- Так легче продемонстрировать возвращение null.
- Но для надёжности P2025 всё равно обрабатывают.

Controller UPDATE

```javascript
export async function updateProduct(req, res, next) {
  try {
    const id = Number(req.params.id);
    const product = await updateProductById(id, req.body);
    if (!product) return res.status(404).json({ message: "Product not found" });
    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

- Перед вызовом требуется middleware валидации id и body.
- Вариант согласован с Service, который возвращает null при отсутствии объекта.

Метод delete()

- Удаляет одну запись по уникальному условию where.
- Возвращает объект удалённой записи.
- Может выдать P2025, если товара нет.
- Внешние ключи могут запретить удаление товара, который участвует в других таблицах.

Удаляем Product

```javascript
const product = await prisma.product.delete({
  where: { id: 15 }
});
```

Что важно

- Prisma выполняет DELETE в PostgreSQL.
- Возвращённые данные можно отправить клиенту.

Delete с проверкой существования

```javascript
export async function deleteProductById(id) {
  const existing = await prisma.product.findUnique({
    where: { id }
  });
  if (!existing) return null;
  return prisma.product.delete({ where: { id } });
}
```

Что важно

- Вспомогательная проверка нужна для старой логики Controller.
- Даже после проверки возможен P2025.

Слайд №39

DELETE /api/products/:id → prisma.product.delete()

Express.js + Prisma + PostgreSQL

Удаление товара через Prisma Client

Клиент



DELETE /api/products/15

Controller

productController.js

```
export async function
 deleteProduct(req, res, next) {
  try {
   const id = Number(req.params.id);
   const product = await
   deleteProductById(id);

   if (!product) {
    return next(new AppError(
     "Product not found", 404));
    });
   }
   res.json(product);
  } catch (error) {
   next(error);
  }
}
```

Service

productService.js

```
export async function
 deleteProductById(id {
  const existing = await
   prisma.product.findUnique({
    where: { id }
  });

  if (!existing) {
   return null;
  }

  return prisma.product.delete({
   where: { id }
  });
}
```

Prisma Client

```
await prisma.product.delete({
  where: { id: 15 }
})
```

PostgreSQL

Таблица products
(до удаления)

id	name	price	...
15	Laptop	42000	...

Удаление строки
с id = 15

Таблица products
(после удаления)

id	name	price	...

(нет строки с id = 15)

Результат (успешно)

200 OK

JSON-ответ
(удалённый объект)

```
{
  "id": 15,
  "name": "Laptop",
  "price": 42000,
  "categoryId": 1,
  "inStock": true
}
```

Товар не существует → DELETE /api/products/999 →

```
prisma.product.findUnique({
  where: { id: 999 }
})
// возвращает null
```

→ return null →

404 Not Found

```
{
  "error": "Product not found"
}
```

Важно: Сначала проверяем существование товара. Если Product найден — удаляем и возвращаем удалённый объект. Если Product не найден — возвращаем 404 Not Found.

DELETE физически удаляет строку из таблицы products в PostgreSQL.

Controller DELETE

```javascript
export async function deleteProduct(req, res, next) {
  try {
    const id = Number(req.params.id);
    const product = await deleteProductById(id);
    if (!product) return res.status(404).json({ message: "Product not found" });
    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

- HTTP-слой возвращает объект удалённого товара.
- Обработка ошибок передана следующему middleware.

P2025 и HTTP 404: единый errorHandler

```javascript
app.use((err, req, res, next) => {
  if (err.code === "P2025") {
    return res.status(404).json({
      message: "Product not found"
    });
  }
  console.error(err);
  res.status(500).json({ message: "Server error" });
});
```

Что важно

- Если Service вызывает update/delete без предварительного SELECT, P2025 преобразуется в 404.
- Размещать обработчик нужно после Router.
- Другие ошибки Prisma нельзя автоматически считать 404.

Полный Product Service

Меняем механизм доступа к данным, сохраняя HTTP API

Read-операции Product Service

```javascript
export async function getAllProducts() {
  return prisma.product.findMany({
    orderBy: { id: "asc" }
  });
}

export async function findProductById(id) {
  return prisma.product.findUnique({
    where: { id }
  });
}
```

Что важно

- Service возвращает JavaScript-данные.
- Controller не зависит от способа обращения к PostgreSQL.

Create-операция Product Service

```javascript
export async function addProduct(data) {
  return prisma.product.create({
    data: {
      name: data.name,
      price: data.price,
      categoryId: data.categoryId,
      inStock: data.inStock ?? true
    }
  });
}
```

Что важно

- Получаем проверенный data без req/res.
- categoryId — обязательная внешняя связь.

Update и Delete в Product Service

```javascript
export async function updateProductById(id, changes) {
  return prisma.product.update({
    where: { id },
    data: changes
  });
}

export async function deleteProductById(id) {
  return prisma.product.delete({
    where: { id }
  });
}
```

Что важно

- id и changes проверены заранее.
- Отсутствующие записи обработает общий errorHandler по коду P2025.

SQL и Prisma: итоговое соответствие

Раньше (pg)	Теперь (Prisma)
SELECT ... FROM products	prisma.product.findMany()
SELECT ... WHERE id = $1	prisma.product.findUnique()
INSERT ... RETURNING	prisma.product.create()
UPDATE ... RETURNING	prisma.product.update()
DELETE ... RETURNING	prisma.product.delete()

Слайд №45

Полный CRUD Product API через Prisma

Node.js + Express.js + Prisma + PostgreSQL

Четыре REST-операции, одна архитектура, без ручного SQL

REST API (HTTP)

- POST /api/products — Создать товар
- GET /api/products — Получить список товаров
- GET /api/products/:id — Получить товар по id
- PATCH /api/products/:id — Изменить товар
- DELETE /api/products/:id — Удалить товар

JSON (новый товар) →
Параметры (id) →
JSON (изменения) →
Параметры (id) →

Product Controller

Принимает HTTP-запросы

Проверяет данные (Validation)

Вызывает методы сервиса

Формирует HTTP-ответы (статусы и JSON)

Product Service

Содержит бизнес-логику
Использует Prisma Client
Больше нет pool.query()
Никаких ручных SQL-запросов

- addProduct(data)
- getAllProducts()
- findProductById(id)
- updateProductById(id, changes)
- deleteProductById(id)

Prisma Client

Типобезопасные запросы к базе данных

```
prisma.product.create({
  data: ...
})
```

```
prisma.product.findMany()
prisma.product.findUnique({
  where: { id: ... }
})
```

```
prisma.product.update({
  where: { id: ... },
  data: ...
})
```

```
prisma.product.delete({
  where: { id: ... }
})
```

PostgreSQL

Хранилище данных (таблица products)

products

id	name	price	...
1	Laptop	42000	...
2	Mouse	3500	...
3	Monitor	28000	...
...	...	...	...

SQL исчезает из Service

Больше нет pool.query(), ручных SQL-запросов и строк с SQL. Все операции выполняются через Prisma Client.

PostgreSQL остаётся

Prisma продолжает работать с вашей существующей базой данных. Все изменения реально сохраняются в PostgreSQL.

Те же принципы REST

Архитектура приложения не меняется:
Routes → Controller → Service → Database.
Мы заменили только способ доступа к данным.

Не меняем маршруты — сохраняем поведение API

- GET /api/products и GET /api/products/:id продолжают возвращать 200 или 404.
- POST /api/products возвращает 201 и созданный объект.
- PATCH и DELETE используют существующие маршруты и HTTP-правила.
- Если прежний API возвращал строковую category, её нужно сформировать из category.name.

Сохраняем прежний формат JSON

```javascript
function formatProduct(product) {
  if (!product) return null;
  const { category, ...data } = product;
  return { ...data, category: category.name };
}

const products = await prisma.product.findMany({
  include: { category: true }
});
return products.map(formatProduct);
```

Что важно

- include загружает название категории из связанной таблицы.
- В актуальной БД category обязательна, categories.name присутствует.
- Не добавляйте вызов Prisma в Controller: преобразование ответа выполняет Service.

Проверяем изменения в Prisma Studio

BASH

```
npx prisma studio
```

Что важно

- После POST появилась новая строка.
- После PATCH обновилось значение price.
- После DELETE строка удалена.
- Основная проверка ЛР — через REST API.

Проверка HTTP-ошибок

Ситуация	Ожидаемый HTTP	Где обрабатываем
Некорректный name/price/categoryId	400	Validation middleware
GET несуществующего товара	404	Controller
PATCH/DELETE отсутствующего товара	404	Prisma P2025 → errorHandler
Внутренняя неизвестная ошибка	500	Общий errorHandler
Несуществующий FK categoryId	4xx по правилам API	Отдельная обработка ограничения внешнего ключа

Мини-практика: полный CRUD через Prisma

Задание

- Перепишите Product Service без pool.query().
- Сохраните Router и Controller.
- Сохраните валидацию HTTP body и id.
- Проверьте 5 HTTP-операций и статус 201.
- Проверьте 400 и 404.
- Сверьте изменения через Prisma Studio.

Критерии проверки

- Все операции выполняются через Prisma Client.
- Записи физически меняются в PostgreSQL.
- Контроллеры не содержат Prisma-запросов.
- Ответы REST API совместимы с прежними.
- Нет ошибки 500 для некорректных входных данных.

Контрольные вопросы

1. Как соотносятся POST и create()?
2. Чем findMany() отличается от findUnique()?
3. Что означают where и data?
4. Для чего нужны select и include?
5. Почему categoryId проверяют перед созданием?
6. Когда findUnique() возвращает null?
7. Что такое ошибка P2025?
8. Почему validation middleware остаётся необходимой?
9. Как сохранить прежний JSON при переходе на Prisma?
10. Что меняется в архитектуре Service → Database?

Итоги занятия

1. Prisma предоставляет методы CRUD без ручного SQL.
2. findMany и findUnique реализуют чтение.
3. create добавляет данные, update меняет, delete удаляет.
4. where, data, orderBy, select и include управляют запросом.
5. Validation возвращает 400 до обращения к базе.
6. Отсутствующие записи преобразуются в 404.
7. Route и Controller сохраняются; меняется Product Service.